A Joint LEAN4-and-Julia Charter for the FSA + `smc2fc` Stack

Part I: LEAN4 as the formal source of truth for FSA models.
Part II: Julia as the production framework for inference and control.
Together: a typed pipeline from LaTeX equations to running code,
with no Python pattern-matching surface for LLM agents to fall through.

Ajay Talati + Claude (post-mortem author)

May 6, 2026

Abstract

This document is a two-part post-mortem-and-charter for a project that has been burned once too often by LLM coding agents pattern- matching their way through Python+JAX scientific code. **Part I** records the failure mode that made the FSA-v5 model implementation and its import into the `smc2fc` framework a multi-day, expensive process: *LLM coding agents pattern-match on variable names instead of building a semantic model of types, roles, dimensions, and pipeline position.* It prescribes the structural fix on the model side: LEAN4 as the formal source of truth, with Python as a secondary, differentially-tested artefact. **Part II** extends the same diagnosis to the `smc2fc` framework itself — also Python, also agent-touched, also exposed to the same pattern-matching surface — and prescribes the matching structural fix: a Julia port optimised for Julia idioms (multiple dispatch, parametric types, native loops, `AbstractArray` dispatch), with both CPU and GPU backends in scope from day one. Part II is explicitly *not* a Python-in-Julia re-skin; an idiomatic Julia codebase is the deliverable, and the work amounts to roughly seven hours of focused translation between two scientific computing languages. The two parts together form a single typed pipeline: LaTeX equations → LEAN4 reference (model side) → Julia framework (`smc2fc` side), with Python remaining in service as the differentially-tested fast path. This document exists so that no future iteration relitigates the question.

Contents

I LEAN4-First Charter for FSA Model Development 3

1 The problem in one paragraph 3

2 Concrete evidence 3
- 2.1 Variable-name collisions in the live code . . . 3
- 2.2 The Bug 5 incident . . . 4
- 2.3 Plan churn in `claude_plans/` . . . 4
- 2.4 Errata after deploy . . . 5

3 Root-cause analysis 5

4 The structural fix: LEAN4 as the source of truth 6
- 4.1 Dependent types make the collision a compile error . . . 6
- 4.2 LEAN4 is also a programming language . . . 6

4.3 mathlib already has the infrastructure . . . 6
4.4 What this does not solve . . . 6

5 The new development process **6**

6 Scope and timing (model side) **7**

7 What is and isn't covered (model side) **8**

8 Audit checklist (model-side agent) **8**

9 Closing of Part I **8**

II Julia Port Charter for the `smc2fc` Framework **10**

10 The problem in one paragraph (framework side) **10**

11 The Python + LLM perfect storm **10**
11.1 Everything is `jnp.ndarray` . . . 10
11.2 Mutability and pure-functional code coexist invisibly . . . 11
11.3 Closures over data are a recompile landmine . . . 11
11.4 The vmap/while-loop nesting cliff . . . 11

12 Why Julia narrows the failure surface **11**
12.1 Multiple dispatch + parametric types . . . 11
12.2 Functional style is the default . . . 11
12.3 Code reads like the LaTeX . . . 11
12.4 No two-language problem . . . 12
12.5 Static analysis catches the rest . . . 12
12.6 LEAN4 alignment . . . 12

13 Existing Julia scaffolding — what we re-use **12**

14 Hybrid CPU/GPU execution strategy **14**
14.1 The two phases of SMC^2 have opposite hardware profiles . . . 14
14.2 The Julia hybrid pipeline . . . 14
14.3 Why this was infeasible in Python+JAX . . . 15
14.4 Architectural type boundaries . . . 15
14.5 What this does to the NUTS question . . . 15

15 The port plan — file-by-file mapping **16**
15.1 Strategic principle: optimise for Julia, not a Python re-skin . . . 16
15.2 Phase 1 — Foundations (~1 hour) . . . 18
15.3 Phase 2 — Filtering (~2 hours, GPU-resident) . . . 18
15.4 Phase 3 — Outer SMC^2 (~1.5 hours, CPU-resident) . . . 19
15.5 Phase 4 — Control (~1 hour) . . . 19
15.6 Phase 5 — Plant + Simulator (~0.5 hour) . . . 19
15.7 Phase 6 — End-to-end + GPU benchmark (~1.5 hours) . . . 19

16 Scope and timing (framework side) **20**

17 What is and isn't covered (framework side) **20**

18 Audit checklist (framework-side agent) **21**

19 Closing of Part II **22**

Part I
LEAN4-First Charter for FSA Model Development

Part I scope note

What follows in Part I is the standalone LEAN4 charter, included here verbatim so this document is self-contained for an agent who has not read the original at `FSA_model_dev/LaTex_docs/lean4_first_charter.pdf`. Part I addresses the **model side**: how new mathematical models (FSA-v5 today, future variants tomorrow) are specified and verified before any Python implementation is written. Part II (below) extends the same discipline to the `smc2fc` framework that consumes those models.

1 The problem in one paragraph

The current FSA development cycle is: write the mathematical model in LaTeX; ask an agent to port it to Python; spend days hunting bugs the agent introduced; eventually ship, often with errata documented post-deployment. The bottleneck is *agent pattern-matching*, not modelling complexity. A junior human collaborator with the same LaTeX source would not make the same class of mistake, because they would read the equations, internalise that $\theta_{\text{ctrl}} \in \mathbb{R}^{16}$ and $\theta_{\text{dyn}} \in \mathbb{R}^{37}$ are different objects, and refuse to compare them as scalar intervals. LLM agents reach for the strongest training association of the name `theta` and write the most-likely-looking code. The fix has to be structural — something that makes the wrong code refuse to compile — not motivational.

The audience for this document is two-fold: (a) the user, as a personal accountability artefact recording why the LEAN4-first process is being adopted and at what point; and (b) every future Claude (or other LLM) coding agent that touches an FSA model, who must read this charter at session start and follow the process it prescribes.

2 Concrete evidence

This section anchors the post-mortem in named, citable incidents. No vague complaints; every claim has a file path or commit hash attached.

2.1 Variable-name collisions in the live code

The same identifier `theta` refers to two fundamentally different mathematical objects in two parts of the codebase, and each carries a different dimension, role, and domain.

(a) `theta` as 16-D RBF schedule coefficients (controller decision variable). In `models/fsa_high_res/control.py`, lines 61–65:

```
def schedule_from_theta(theta: jnp.ndarray) -> jnp.ndarray:
    theta_B = theta[:n_anchors]
    theta_S = theta[n_anchors:]
    raw_B = c_Phi + jnp.einsum('a,ta->t', theta_B, Phi_design)
```

Here `theta` is a length-16 vector ($2 \times n_{\text{anchors}}$, with $n_{\text{anchors}} = 8$), interpreted as RBF basis coefficients that, after a sigmoid envelope, parameterise the dual-channel control schedule $\Phi(t)$. It is the controller's *decision variable*. It lives in $\mathbb{R}^{16}$. Its prior breadth is the scalar `sigma_prior` in `smc2fc/control/config.py:28`.

(b) `theta` as 37-D dynamics-parameter posterior (SMC^2 inference target). In `models/fsa_high_res/estimation.py`, the function `get_init_theta()` at line 513–520:

```
def get_init_theta():
    """Return prior-mode parameter vector (length = len(PARAM_PRIOR_CONFIG))."""
    return np.array([_mode(pt, pa) for _, (pt, pa)
                     in PARAM_PRIOR_CONFIG.items()],
                    dtype=np.float32)
```

Here `theta` is a length-37 vector of physiological parameters — rate constants, amplitudes, decay times, observation noise scales — whose posterior the SMC^2 machinery infers from data. It is the inference *target*, not a decision variable. It lives in $\mathbb{R}^{37}$. Its prior is component-wise log-normal, not Gaussian; the prior breadth varies per parameter.

Why this matters. An agent reading either file sees `theta`, a flat numpy array, and reaches for "Gaussian prior on θ" — which is correct in case (a) and wrong in case (b). An agent reasoning across files conflates the two and treats "the prior on θ" as a single concept with a single dimension and a single breadth. There is no warning sign in the Python code itself; the type is just `jnp.ndarray` in both places.

2.2 The Bug 5 incident

The named incident that motivated this charter, recorded verbatim from the user's feedback memory at `~/.claude/projects/.../memory/feedback_no_fabricated_bugs.md`:

> *During the FSA-v5 verification session on 2026-05-06 I logged a "Bug 5" that claimed $\theta \in [-0.4, +0.5]$ for the controller's RBF schedule prior to keep the system in the healthy island. θ is a 16-dim vector (RBF coefficients across both Phi channels and* ***`n_anchors=8`*** *anchors). The healthy island is a 2D region in (Φ_B, Φ_S)* ***control output space****, not in θ space. Comparing them as scalar intervals is incoherent. I conflated:*
>
> - *SMC^2 parameter posterior θ (37-dim, the inferred parameters)*
> - *Controller RBF schedule θ (16-dim, the decision variables)*
>
> *both of which use* ***`sigma_prior`*** *in the bench code, mentally collapsed to a single scalar. The bench code in front of me had* ***`theta_B = theta[:n_anchors]`*** *— explicit vector indexing — and I didn't internalize it.*

The agent in question was me. The user's verdict was "stupid mistake"; the request was for guardrails. This document is the guardrail.

2.3 Plan churn in `claude_plans/`

The repository's `claude_plans/` directory accumulates one markdown file per plan-mode session. The CLAUDE.md convention is that meaningful updates to a plan add a `> Updated:` audit line below the title. As of this writing the directory contains eight plan files, of which *seven have zero* ***`Updated:`*** *lines* — they were drafted, never executed, never closed:

- `FSA_v5_Implementation_Plan_for_the_fsa_high_res_Folder_2026-05-04_2238.md` (0 updates).
- `Lyapunov_Stability_Analysis_for_FSA_v4_Plan_2026-05-04_1448.md` (0 updates).
- `AjaysNotesPlantoupdatetoversion5.md` (0 updates).

- `Advice_on_smc2fc_Port_Cost_Function_Architecture_2026-05-05_1055.md` (0 updates — advice-only, never tied to a closed task).
- `Techicaldocumentationplan-FSA_version_5_technical_guidetex.md` (0 updates).
- `AjaysNotes-1.md`, `AjaysNotes-2.md` (0 updates).

The single plan that did execute is `FSA_dev_sandbox_mirror_the_SWAT_model_dev_pattern_v4_branch_2026-05-04_2353.md` (1 update line, 13/13 tests green at completion). The ratio of abandoned to executed plans (7:1) is not a sign of laziness; it is a sign that planning sessions repeatedly re-derive the same mathematical model in slightly-different shapes because no formal artefact pins the model down.

2.4 Errata after deploy

Commit `d886f36` on the v4 branch:

> *docs: add FSA-v4 errata, open-loop limitation, and SMC^2 vs OT comparison.*
>
> *Documents the gap between the §4 narrative and the actual behaviour of `tools/optimize_dynamic_plan_`*
> *then frames smc2fc as the structural remedy.*

A documentation-correction commit recording bugs caught *after* deployment, not during development. The fact that errata was needed at all is the evidence that the in-development verification was not strong enough; it does not matter how many tests pass on the final commit if the bugs are documented in a follow-up commit instead of caught at the place they were introduced.

3 Root-cause analysis

LLMs are surface pattern-matchers. They read source code primarily as text, not as a typed graph of objects with roles. When an agent sees `theta`, the most-trained association fires; the surrounding context is treated as secondary. This is not a defect of any particular model; it is the structural consequence of how the system is built. Asking an agent to "be more careful" is asking it to override its own training prior every time — which empirically does not happen reliably, as §2.2 demonstrates.

Python provides no formal scaffolding to compensate. `jnp.ndarray` is the type of both the 16-D control coefficient vector and the 37-D parameter posterior. There is no shape-aware static analyser running in the loop, no role tagging in the type system, no proof obligation attached to a function signature that the inputs satisfy any pre-condition. The senior engineer (the user) ends up as the only line of defence, and the £££ in tokens is the cost of the user catching agent errors after the fact.

Why "tell agents to be more careful" doesn't work. The user's feedback memory at `feedback_no_fabricated_bugs.md` encodes a PRIMARY RULE — *build a semantic model BEFORE any claim* — which is correct as a principle but unenforceable as a process. The rule depends on the agent *choosing* to think rather than pattern-match. §2.2 is the empirical proof that the choice is not reliably made. Every agent session is one roll of the dice on whether the rule fires that day.

4 The structural fix: LEAN4 as the source of truth

4.1 Dependent types make the collision a compile error

A LEAN4 function with the signature

$$\texttt{evaluateCost} : \texttt{RBFCoeffs}\ 16 \to \mathbb{R}$$

syntactically refuses any value not of the exact type `RBFCoeffs 16`. The 37-dimensional parameter posterior θ_{dyn} : `ParameterVector` 37 is a different type, and `lake build` will reject any code that conflates them. Pattern-matching on names becomes harmless because the type system catches the consequence at compile time, before any incoherent reasoning reaches the user.

The two-dimensional output region (Φ_B, Φ_S) from §2.2 would, in LEAN4, have a type like `ControlOutput` that is *not* the type of any prior on θ. Writing $\theta \in [-0.4, +0.5]$ would not compile, full stop, because the right-hand side is a scalar interval and the left-hand side is a vector. The agent could not have made the mistake.

4.2 LEAN4 is also a programming language

LEAN4 compiles to native code via C. `lake build` produces a binary; a thin Python `ctypes` wrapper exposes each verified function as a Python callable. The same LEAN4 file is both the formal specification and the executable reference implementation. There is no "the spec drifted from the code" failure mode because the spec *is* the code.

4.3 mathlib already has the infrastructure

The mathematical objects FSA-v5 needs — finite-dimensional real vector spaces, matrix operations, exponentials and logarithms, Hill functions, sigmoids, piecewise-linear schedule decoders, ODE integration — are all already library imports in `mathlib`. There is no novel mathematics to formalise. Transcribing the FSA-v5 LaTeX into LEAN4 is genuinely just translation.

4.4 What this does not solve

- *Floating-point semantics.* The gap between the LEAN4 notion of $\mathbb{R}$ and IEEE-754 double-precision arithmetic is real and not addressed by this charter; it is bounded by the tolerance threshold in the differential test (§5 step 6).
- *GPU correctness.* The differential test runs the Python implementation on whatever backend it normally uses (JAX/XLA, GPU); the LEAN4 binary is CPU-only. Disagreements driven by GPU-specific behaviour are out of scope for the LEAN4 reference itself and trusted as before.
- *The transcription itself.* If the LaTeX→LEAN4 transcription is wrong, both the LEAN4 reference and (likely) the agent-written Python will be wrong in the same way. This is why the LaTeX→LEAN4 step is human-reviewed line-by-line; it is the one place reviewer attention has to land.

5 The new development process

The standing rule going forward, and the charter every future agent must follow:

> **For every new mathematical model, every model extension, and every import into `smc2fc`, the LEAN4 specification is written before any Python**

implementation. The LEAN4 native binary is the authoritative reference. Python is differentially tested against it. Disagreement beyond a tolerance threshold is by construction a Python bug.

Per function, the workflow is:

1. **LaTeX equations (as today).** Drift, diffusion, cost, plant, schedule decoder, observation model — written in `LaTex_docs/sections/` using the existing notation conventions.

2. **LEAN4 transcription.** A new `lean/` directory in the repo. The LaTeX equations are transcribed into LEAN4 *definitions* (typed, with dimensions encoded in the type) and *implementations* (runnable code). Line-by-line, side-by-side with the LaTeX. Straight translation, no design work, no novel mathematics.

3. **LEAN4 native compilation.** `lake build` produces a native binary. A Python `ctypes` wrapper in `lean/python_bridge/` exposes each function as a Python callable.

4. **Python implementation (agent-written, fast path).** The agent writes the JAX/Python implementation as today, optimised for GPU. The agent is permitted to pattern-match; step 6 catches the consequences.

5. **(Optional) property proofs.** Lipschitz, basin-of- attraction theorems, monotonicity — the kind of statement that makes the LEAN4 spec a *proof* not just a typed transcription. Defer unless a specific incident motivates one. The charter does not gate on these.

6. **Differential testing.** A `pytest` harness using `hypothesis` for random-input generation evaluates BOTH the LEAN4 binary AND the Python implementation on a battery of inputs sampled from the physiological-bound box. Any disagreement beyond a tolerance threshold (default 10^{-6} in single-step drift, 10^{-4} in integrated trajectory) is a Python bug.

7. **`smc2fc` port.** The port is a thin shim from the verified LEAN4 reference into the framework's `EstimationModel` / `SDEModel` interfaces. The shim is also differentially tested against the LEAN4 binary before being merged.

6 Scope and timing (model side)

This is translation work, not research. The LaTeX spec already exists; LEAN4 has the linear-algebra and ODE infrastructure; `mathlib` supplies the lemmas. The full FSA-v5 model is **one day's work for a competent human developer**. Any estimate quoting weeks per function is inflation; the user has explicitly rejected that framing and this document is bound by the rejection.

Function	File	LEAN4 transcription
FSA-v5 drift	`models/fsa_high_res/_dynamics.py`	$\sim$ 2 hours
Cost (chance-constrained)	`models/fsa_high_res/control_v5.py`	$\sim$ 1.5 hours
StepwisePlant	`models/fsa_high_res/_plant.py`	$\sim$ 1 hour
Schedule decoder (RBF)	`models/fsa_high_res/control.py`	$\sim$ 30 min
Observation model	(in `estimation.py`)	$\sim$ 1 hour
FFI wrapper + diff tests	`lean/python_bridge/`	$\sim$ 2 hours
Total	—	$\sim$ 8 hours

The LEAN4 source is roughly the same length as the LaTeX it transcribes. Most of the time goes into wiring up the FFI bridge and writing the differential-test harness; the math itself is direct.

The cost/value frame: a single 8-hour LEAN4 transcription pass amortises across *every future FSA version, every future model extension, and every future agent attempting a port to* `smc2fc`. The alternative — paying for repeated agent rounds of pattern-match-then-hunt-bugs — is what burned the £££ this charter exists to stop.

7 What is and isn't covered (model side)

In scope. Drift, cost, plant, schedule decoder, observation model. Typed LEAN4 reference implementations plus differential tests. The agent-written FSA-v5 surface in `models/fsa_high_res/` (2,200 Python LOC across 4 files; the LEAN4 transcription is much shorter).

Out of scope (still trusted). The SMC^2/PF framework code in `smc2fc/core/` and `smc2fc/filtering/` is already tested upstream and is not subject to *this Part*. **Note:** Part II below proposes a Julia port of the framework that does subject this code to type-safety scrutiny by structural means; that is the joint deliverable of this combined charter. HMC and NUTS internals, the JAX-native compile-once architecture, GPU correctness, and IEEE-754 floating-point semantics are trusted as before.

Trusted by transcription. The LaTeX→LEAN4 step is human-reviewed line-by-line. This is the one place reviewer attention has to land; the rest of the process is mechanical.

8 Audit checklist (model-side agent)

A future agent runs this checklist at session start, before touching any FSA-related Python file:

- ☐ Is there a LEAN4 reference for the function I am about to touch? *If no:* the charter says I am not allowed to write the Python until the LEAN4 reference exists. Stop and flag.
- ☐ Does `lake build` succeed on the current commit (the LEAN4 reference type-checks)?
- ☐ Does the Python differential test pass against the LEAN4 binary on the current commit (`pytest tests/test_lean4_diff.py` is green)?
- ☐ Have I written down — in plain English — the type, role, and pipeline position of every variable I am about to reason about? (Per the PRIMARY RULE in `feedback_no_fabricated_bugs.md`.)
- ☐ If I am about to claim a bug: do I have a reproducer (input → both implementations → disagreement beyond the tolerance threshold)? *If no:* it is not a bug, it is an open question; record it as such and move on.

If any box is unchecked, the agent does not proceed. The session is paused; the user is notified; the missing artefact is produced before any code is changed.

9 Closing of Part I

The point of Part I is accountability on the model side. The named incident in §2.2 was caused by an agent (me) pattern- matching on `theta` across two architectural layers it does not unify. The cost was burned trust and burned time. The charter prescribed in §5 makes that class of error structurally impossible: the LEAN4 type-checker is unforgiving, and disagreements between LEAN4 and Python are by construction Python bugs.

LEAN4 is never wrong about types. Python is. Agents pattern-match; LEAN4 does not. Part I is the bridge that makes the agent's limitations harmless on the model side.

Bridge to Part II. The LEAN4 charter solves half of the problem — the model side. What about the framework `smc2fc` itself? It is also Python, also agent-touched, also exposed to the same pattern-matching surface, and the empirical evidence in §2 (the `theta` 16-D vs 37-D collision, the closure-of-data recompile bug, the NUTS-inside-vmap-inside-`lax.while_loop` cliff) is at least as pointed about `smc2fc` internals as it is about `models/fsa_high_res/`. Part II prescribes the matching structural fix for the framework side.

Part II
Julia Port Charter for the `smc2fc` Framework

10 The problem in one paragraph (framework side)

The `smc2fc` codebase is doing the right mathematics; the failure mode is the same as the FSA-v5 model side: agent pattern-matching across a Python codebase with no type-level enforcement of dimensions or roles. Part I solves half of this — the model side — by making LEAN4 the formal source of truth for the math `smc2fc` consumes. Part II solves the other half by porting the framework itself to a language where the type system catches the pattern-matching errors at compile time. The audience is the same two populations: the user (accountability) and future LLM agents (constitution loaded at session start).

11 The Python + LLM perfect storm

Python and LLM coding agents combine into a particularly bad failure surface for the kind of math `smc2fc` does. This section names the four failure modes inside `smc2fc/` specifically.

11.1 Everything is `jnp.ndarray`

Python+JAX has one type for every numerical object regardless of shape, role, or origin. Concrete signatures from the live code:

(a) `smc2fc/core/jax_native_smc.py:67` —

```
def _ess_at_delta(loglikelihood_fn, particles, delta):
    ...
```

`particles` is a 2-D array of shape `(n_smc, d)`; the type annotation is just `jnp.ndarray`. An agent passing a single particle (a 1-D array of shape `(d,)`) gets a runtime broadcast surprise — the function returns a wrong-shape result silently.

(b) `smc2fc/filtering/_gk_kernel.py:185` —

```
def smooth_resample_basic(particles, log_weights, ...):
    ...
```

`log_weights` could be a 1-D vector of length `n_pf` (if the caller is operating on a single particle filter) or a 2-D `(n_smc, n_pf)` block (if the caller has already `vmap`'d over the SMC^2 outer cloud). Nothing in the signature says which. The function happens to work in both cases because of broadcasting; an agent reasoning about either case internalises one of the two and writes downstream code that breaks in the other.

(c) `smc2fc/core/tempered_smc.py` — the BlackJAX-wrapped backend captures `data` in a closure per window. This is the bug §6 of the framework documentation documents: per-window recompiles dominate the runtime. The Python+ JAX foible is invisible at the call site; an agent reading the function signature has no way to predict that `data` ends up hashed into the JIT cache key.

11.2 Mutability and pure-functional code coexist invisibly

In `smc2fc/control/diagnostics.py` the `matplotlib` helpers are imperative side-effectful Python; in `smc2fc/core/jax_native_smc.py` the SMC loop is pure-functional for JIT compatibility. Both files use the same `def` keyword, the same `jnp.ndarray` types, the same import style. An agent editing one file reaches for the pattern of the other and silently breaks the JIT contract — mutating an array inside what was a pure function, capturing a non-traceable Python object inside a JIT'd closure. The LEAN4 charter's PRIMARY RULE ("build a semantic model before any claim") has to fire on every cross-file edit; empirically it does not.

11.3 Closures over data are a recompile landmine

A long-running session that thinks it is reusing a JIT-compiled function silently re-traces because a dataclass's `__hash__` changed. Documented in framework-doc §6.1; an agent cannot infer this from any signature in the file because the cause lives in JAX's tracing semantics, not in the Python source.

11.4 The vmap/while-loop nesting cliff

§6.4 of the framework documentation records that NUTS-inside- `vmap`-inside-`lax.while_loop` is 60–85× slower than HMC because of dynamic-tree-building inside nested control flow. This is a Python+JAX architectural consequence, not a mathematical one — and it is exactly the kind of trap an agent walks into by pattern-matching "NUTS is a strict upgrade from HMC" from training data. The signature `nuts.build_kernel(...)` says nothing about the nesting cliff; the agent has to know JAX's compilation model to predict the disaster.

12 Why Julia narrows the failure surface

12.1 Multiple dispatch + parametric types

A Julia function declared

```
function ess_at_delta(loglik::F, particles::Matrix{Float64},
                      delta::Float64) where {F<:Function}
    ...
end
```

refuses anything else at compile-time. `Vector{Float64}` and `Matrix{Float64}` are different types; passing one where the other is expected does not silently broadcast. The single-particle vs particle-cloud confusion from §11.1(a) becomes a method-not-found error at the call site, not a wrong-shape result downstream.

12.2 Functional style is the default

Mutation is opt-in via the `!` suffix convention: `resample!(particles, weights)` mutates in place; `resample(particles, weights)` returns a fresh array. An agent reading any function signature can tell instantly which mode it is in. The Python invisibility from §11's "imperative diagnostics file vs pure-functional SMC loop file" disappears.

12.3 Code reads like the LaTeX

A Julia drift function with parametric types reads almost identically to the equation it transcribes:

```
function drift(x::FSAState{6}, theta::DynParams)::FSATangent{6}
    ...
end
```

This is much closer to the LEAN4 spec from Part I than the equivalent Python. An agent moving between LEAN4 and Julia is moving between two flavours of typed dispatch; an agent moving between LEAN4 and Python is moving between a typed system and an untyped one and has to translate types out of sight at every step.

12.4 No two-language problem

Python+JAX exists because Python is too slow for the inner loop; JAX is the workaround that ships the inner loop to XLA. Julia is fast natively. The JIT-traced / non-traced split that creates the invisible mutability landmine of §11 is a *Python* artefact; in Julia, the inner and outer loops are the same kind of code.

12.5 Static analysis catches the rest

`JET.jl` runs a static type analysis on the entire package and flags every type uncertainty. This is a single command:

```
julia --project=. -e 'using JET; report_package(SMC2FC)'
```

There is no analogue in Python (`mypy` doesn't reach into JAX trees, and even where it does, the pervasive `jnp.ndarray` type defeats it).

12.6 LEAN4 alignment

A LEAN4 specification for the FSA model (per Part I) translates to Julia almost one-to-one because both languages are dispatch-based and type-driven. The LEAN4 → Julia transcription is shorter than LEAN4 → Python by roughly the ratio of the language verbosities. The combined LEAN4 reference plus Julia framework forms a single coherent typed pipeline *from the LaTeX equations to the running code*. Python sits awkwardly in the middle of this pipeline today; remove it from the framework side and the pipeline aligns.

13 Existing Julia scaffolding — what we re-use

The bulk of `smc2fc/` is wiring well-known algorithms together. Julia already has each of those algorithms as a maintained library; the port is mostly *replacing custom code with library calls*. The table below is the per-module mapping.

Python `smc2fc` module	Julia replacement	Notes
`core/jax_native_smc.py` HMC kernel	`AdvancedHMC.jl`	The original HMC implementation Black-JAX modelled on. Direct drop-in.
`core/tempered_smc.py`	Custom (built on `AbstractMCMC.jl`)	No exact analogue but the abstractions are there; ∼ 150 lines of Julia.
`core/sf_bridge.py` BW geodesic	`LinearAlgebra` (stdlib) + `Distributions.jl`	Matrix square root, multivariate Gaussian. ∼ 50 lines.
`core/mass_matrix.py`	`Statistics.var` (stdlib)	Literal one-liner.
`filtering/sinkhorn.py` + `transport_kernel.py` + `resample.py:ot_resample_lr`	`OptimalTransport.jl`	Maintained, supports entropic + low-rank.
`filtering/_gk_kernel.py` Liu–West	Custom (∼ 80 lines)	Standard moment-match + Gaussian smoother; trivial in Julia.
`filtering/gk_dpf_v3_lite.py`	Custom (∼ 150 lines)	Bootstrap filter loop; reuses `Distributions.jl` for log-density.
`simulator/sde_solver_diffrax.py`	`StochasticDiffEq.jl` (part of `DifferentialEquations.jl`)	Gold standard. Replaces diffrax wholesale.
`transforms/unconstrained.py`	`Bijectors.jl`	The Julia ecosystem's standard bijection library.

Python `smc2fc` module	Julia replacement	Notes
Einsum-style ops (Liu–West kernels, RBF designs)	`Tullio.jl` or built-in broadcasting	Often broadcasting alone suffices.
Autodiff for HMC gradients	`Enzyme.jl` (preferred) or `Zygote.jl`	`Enzyme.jl` is more efficient for the kind of code here.
Static analysis	`JET.jl`	The "Julia answer to the LLM pattern-match risk".
Testing harness	`Test.jl` (stdlib)	Standard.
GPU arrays	`CUDA.jl`	NVIDIA backend; `CuArray` parametric type plays naturally with the generic `AbstractArray{T,N}` dispatch the rest of the package uses.
Cross-device kernels	`KernelAbstractions.jl`	Write a kernel once, runs on CPU (`CPU()`) or GPU (`CUDABackend()`). Used where broadcasting alone won't suffice.
Performance-critical scaffolding (added per external review):		
Small fixed-size arrays (state, params)	`StaticArrays.jl`	**Critical.** `SVector{6, Float64}` is stack-allocated, zero-overhead, strictly sized. The Julia equivalent of XLA unrolling small arrays into registers. Without it, `State{6}` and `DynParams` would heap-allocate on every step and the inner SDE/SMC loop would regress badly versus JAX.
Particle cloud layout	`StructArrays.jl`	Struct-of-Arrays (SoA) memory layout. A `Particle` struct stored in a `StructArray` reads as `for p in particles` but the underlying memory is contiguous per field; coalesced GPU access.
Parameter blocks	`ComponentArrays.jl`	Flat `AbstractVector` view of nested `DynParams` / `CtrlParams` with dot-access syntax (`theta.k_FB`). Plug-and-play glue between `AdvancedHMC.jl`, `Enzyme.jl`, and the domain types.
Likelihood / posterior interface	`DensityInterface.jl`	Standard wrapper that makes any log-density function plug-and-play with `AdvancedHMC.jl` and the wider Julia MCMC ecosystem.
Particle-filter primitives	`AdvancedPS.jl` (optional)	Battle-tested SMC sweeps, particle-Gibbs samplers, robust resampling schemes from the Turing ecosystem. May replace the $\sim$ 150-line custom bootstrap-filter loop in `src/Filtering/Bootstrap.jl`.
CPU thread parallelism	`Polyester.jl` (or `Base.Threads`)	Replaces JAX's `vmap` on the CPU backend. `Polyester.@batch` has lower overhead than `Threads.@threads` for tight inner loops.

The custom code in the port — the SMC^2 tempering loop, the bootstrap filter wrapper, Liu–West shrinkage, the cost-as-likelihood controller loop, the bridge-warm-start glue — is roughly **1,000–1,200 lines of Julia**. Compare to the $\sim$ 5,600 lines of Python the port replaces. The reduction is mostly the disappearance of JAX-specific boilerplate (closures, `Partial` wrappers, `lax.while_loop` scaffolding, manual `jit` placement).

14 Hybrid CPU/GPU execution strategy

The §11.4 "vmap/while-loop nesting cliff" has a deeper architectural cause that informs how the Julia port should allocate work to hardware. SMC^2 is structurally *heterogeneous*: its inner loop and its outer loop have diametrically opposed hardware preferences, and the Python+JAX implementation could not separate them because JAX's JIT/closure model effectively forces all inside-trace work onto the same device. Julia's per-function compilation lifts that constraint, and the port should exploit it from day one.

14.1 The two phases of SMC^2 have opposite hardware profiles

Inner loop (particle filter, SDE simulation). Massively parallel, SIMD-friendly, predictable control flow. Advancing $N \sim 10^4$ SDE trajectories simultaneously is the textbook GPU workload. Ideal hardware: **NVIDIA RTX 5090** (or equivalent — the dominant cost is memory bandwidth $\times$ CUDA core count).

Outer loop (parameter rejuvenation: HMC / NUTS). Highly branchy, sequential per chain, dynamic control flow — particularly NUTS, where the trajectory length is determined by a U-turn check whose branch pattern varies per particle. GPUs suffer from *warp divergence* when neighbouring threads take different branches; this is exactly what causes the §11.4 60–85$\times$ slowdown. Ideal hardware: **Intel Core Ultra 9 285K** (or equivalent — a strong-IPC, deep-cache, branch-predictor-rich CPU).

14.2 The Julia hybrid pipeline

The Julia port should run the two phases on the hardware they want and connect them with a low-bandwidth interface across the PCIe bus. The data-flow per outer SMC^2 iteration:

1. **Phase A (GPU).** The current parameter cloud $\{\theta^{(m)}\}_{m=1}^{M}$ lives on the host. For each $\theta^{(m)}$, a particle filter with N state particles is run *on the GPU* via `CuArray`-resident state, advancing all particles in parallel via `KernelAbstractions.@kernel` or `StochasticDiffEq.jl`'s GPU dispatch. The output is a scalar $\widehat{L}_N(\theta^{(m)})$ per outer particle.

2. **Phase B (PCIe, summary statistics only).** Only the scalar $\widehat{L}_N(\theta^{(m)})$ (and, if HMC, its gradient with respect to the 37-D $\theta^{(m)}$) is transferred from device to host. Transferring a scalar or a 37-D vector across PCIe Gen 5 takes microseconds — entirely negligible against the SDE integration time. The $N \times d$ particle state matrix is *not* transferred; it stays GPU-resident.

3. **Phase C (CPU).** Tempered SMC^2 outer loop runs on the host. Adaptive $\delta\lambda$ via bisection. Mass-matrix re-estimation. HMC (or, optionally, NUTS) rejuvenation moves via `AdvancedHMC.jl` on the CPU, where the dynamic control flow benefits from branch prediction. The multi-core architecture is exploited by wrapping the outer particle loop in `Threads.@threads` or `Polyester.@batch` — each thread driving its own GPU dispatch for inner work.

4. **Phase D (PCIe, parameter vectors only).** The new parameter proposals $\{\theta'^{(m)}\}$ are sent back across PCIe to the GPU. 37-D vectors $\times M$ outer particles is again negligible bandwidth.

5. **Repeat.**

14.3 Why this was infeasible in Python+JAX

The JAX backend bakes the log-likelihood into a single JIT-compiled closure. Inside that closure, any device split has to be expressed as JAX operations, which means the entire trace must lower to one backend (XLA-CPU or XLA-GPU). Forcing the inner loop to GPU and the outer loop to CPU within the same JIT trace would require host-callback boundaries that defeat the JIT, or a two-program architecture with high-frequency device handoffs that defeat XLA's fusion. Either way, the per-iteration cost of the boundary exceeds the saving from the right-hardware allocation. So the JAX implementation chose "all on GPU", NUTS-inside-vmap-inside- `lax.while_loop`, and paid the §11.4 60– 85× tax.

In Julia, each function compiles independently. The `run_particle_filter!(filter_state::GPUFilterSt theta::CuArray)` method dispatches to a CUDA kernel; the `rejuvenate!(cloud::CPUParameterCloud, log_lik::Vector)` method dispatches to ordinary CPU code calling `AdvancedHMC.jl`. The boundary between them is just two function calls and one `Array(scalar_loglik)` transfer — no JIT trace to keep happy, no monolithic compilation unit to fragment.

14.4 Architectural type boundaries

Two parametric types make the hybrid contract explicit:

```julia
# Particle states: GPU-resident.
struct GPUFilterState{T}
    particles::CuArray{T, 2}      # (n_pf, d_state)
    weights::CuArray{T, 1}        # (n_pf,)
end

# Outer parameter cloud: CPU-resident.
struct CPUParameterCloud{T}
    theta::Matrix{T}              # (n_smc, d_theta), CPU
    log_lik::Vector{T}            # (n_smc,), CPU
end
```

The likelihood function consumed by `AdvancedHMC.jl` handles the host-device transfer internally:

```julia
function compute_likelihood(theta_cpu::AbstractVector{Float64},
                            filter_state::GPUFilterState)
    theta_gpu = CuArray(theta_cpu)              # 37-D, microseconds
    log_lik_gpu = run_particle_filter!(filter_state, theta_gpu)
    return Array(log_lik_gpu)[1]                # scalar back, microseconds
end
```

When `Enzyme.jl` or `Zygote.jl` differentiates this for HMC gradients, the VJP also handles the host-device boundary correctly — the autodiff stays inside whichever device is doing the work and only the gradient w.r.t. `theta_cpu` crosses back.

14.5 What this does to the NUTS question

§11.4 records that NUTS-inside-vmap-inside- `lax.while_loop` is the architecturally infeasible combination, not NUTS itself. Under the hybrid pipeline, NUTS runs on the CPU where its branchy dynamic-tree-building is well- matched to the hardware. The 60–85× slowdown that ruled NUTS out in Python therefore *does not transfer* to the Julia port. The default Julia port still uses HMC (replicating the framework-doc default), but NUTS becomes a viable future-work candidate under §17's "what changes when the Python+JAX architectural constraints are removed" frame.

15 The port plan — file-by-file mapping

The Julia package is named `SMC2FC.jl` and lives at `python-smc2-filtering-control/julia/SMC2FC/` (sister directory to the Python `smc2fc/`). The port proceeds bottom-up so that lower layers are tested before upper layers depend on them.

15.1 Strategic principle: optimise for Julia, not a Python re-skin

The most common mistake an agent will make on this port is translating Python line-by-line. The result is a wooden, Python-shaped Julia package that realises none of Julia's actual advantages — and is therefore not worth the porting effort at all. The point of doing this work is to land in a codebase that is *idiomatic Julia*: shorter, faster, more type-safe, and better matched to the LEAN4 spec language than the Python original. **A "Python in Julia syntax" port fails the charter even if it type-checks and passes the differential test**, because it does not realise any of the §12 benefits.

The translation discipline is therefore to port the *algorithm* and let Julia's native features (loops, multiple dispatch, broadcasting, `AbstractArray` dispatch) express it the Julia way:

Replace JAX control flow primitives with native Julia loops. `jax.lax.scan`, `jax.lax.fori_loop`, `jax.lax.while_loop` exist in JAX because Python's interpreter overhead makes a plain `for` loop unusably slow. **Julia has no such overhead.** A standard `for k in 1:n` loop in Julia is fast under both ordinary execution and `Enzyme.jl` / `Zygote.jl` autodiff. Do not port `lax.scan` to a `Tullio.jl` reduction or to a `KernelAbstractions.jl` kernel out of habit — port it to a `for` loop. The control-flow scaffolding that bloats the JAX source files (the $\sim$ 5,600 $\rightarrow\sim$ 1,200 LOC reduction from §13 comes mostly from this) collapses into ordinary Julia.

Replace conditional logic with multiple dispatch. Where Python branches on flags (e.g. the `bridge_type` switch in `core/sf_bridge.py`, the soft-vs-hard chance-constrained switch in `control_v5.py`), Julia dispatches on a small set of marker types:

```julia
struct GaussianBridge end
struct SchrodingerFollmerBridge end

bridge_init(::GaussianBridge, prev_cloud, new_cloud) = ...
bridge_init(::SchrodingerFollmerBridge, prev_cloud, new_cloud) = ...
```

The conditional logic disappears into method definitions; each behaviour lives in its own method, the dispatcher chooses at compile time, and the agent cannot accidentally reach the wrong branch.

Flatten nesting. Python's `vmap`-over-particles + `lax.while_loop`-over-tempering + `lax.scan`-over-bins nests three primitives because each layer needs its own JAX-recognised iterator. In Julia: nested `for` loops, end of story. Broadcasting (`f.(particles)`) handles the `vmap` layer; ordinary loops handle the rest.

Use `AbstractArray{T,N}` in signatures, not concrete `Array` or `CuArray`. This is the GPU-portability discipline from §13 stated as a translation rule: write the function once against the abstract type, run it on whichever backend the caller supplied. The agent should never need to write two versions of any function.

Zero-allocation inner loop — opt-in mutability where it counts. JAX is pure-functional and relies on XLA to fuse and eliminate the intermediate arrays each step would otherwise allocate. Julia has no such fuser. A naive `for` loop that returns a fresh `x_new = f(x)` array on

every iteration *will run substantially slower than the equivalent JAX* `lax.scan`. The discipline is therefore: keep the *public API* functional (the `!` convention from §12.2 still holds at module boundaries), but in the *inner hot loops*, preallocate the working buffers once and use in-place mutating operations:

```
# Hot SDE step: NO per-iteration allocation.
function step!(x::AbstractVector, dt::Float64, theta::DynParams)
    x .= x .+ dt .* drift(x, theta)
    return x  # x has been mutated in place
end
```

A reasonable rule of thumb: every Phase 2–5 inner loop should be benchmarked with `BenchmarkTools.@btime` and report `0 allocations` per iteration (or a single allocation that is amortised). Anything else is a missed optimisation, not a shippable result.

Explicit CPU multithreading via `Threads.@threads` or `Polyester.@batch`. A subtlety the previous bullets do not capture: JAX's `vmap` parallelises the per-particle work on multi-core CPUs implicitly. A plain Julia `for p in particles ... end` is sequential. To match (or beat) the JAX-on-CPU baseline, outer particle loops have to be wrapped in a threading macro:

```
# Outer SMC loop, CPU multithreaded.
Threads.@threads for m in 1:n_smc
    log_lik[m] = inner_pf_loglikelihood(theta_dyn[m], data)
end

# Or, lower-overhead, for tight loops:
Polyester.@batch for m in 1:n_smc
    log_lik[m] = inner_pf_loglikelihood(theta_dyn[m], data)
end
```

On the GPU backend, `KernelAbstractions.@kernel` replaces both. The agent should not ship any outer-particle loop in serial form on the CPU backend.

Heavy broadcasting can hide allocations — use `@.` or explicit loops in hot paths. The earlier "broadcasting suffices" note is true for simple operations, but chained broadcasting on custom structs can quietly create intermediate temporary arrays unless every operation is fused into a single `@.` block:

```
# Slow: each '.' may allocate a temporary.
x_new = a .+ b .* sin.(c) .- d

# Fast: single fused broadcast, no temporaries.
x_new = @. a + b * sin(c) - d

# Fastest in tight inner loops: explicit loop with @inbounds @simd.
@inbounds @simd for i in eachindex(x_new)
    x_new[i] = a[i] + b[i] * sin(c[i]) - d[i]
end
```

For complex inner-loop particle state transitions, the explicit `@inbounds @simd` form (CPU) or a `KernelAbstractions` kernel (GPU) is safer and usually faster than chained broadcasting.

Type stability is the #1 Julia performance killer — `JET.jl` is the lint that catches it. A function whose return type the compiler cannot infer at compile time falls back to runtime dynamic dispatch on every call. Inside a hot SMC^2 inner loop, a single type-unstable line can cost 10–100× in performance and is invisible at the source level. `JET.jl`'s `report_package` catches this. The audit checklist (§18) requires it green; if it flags a `Union{Float64, Nothing}` return in a hot path, the agent must refactor (e.g. replace `Nothing` sentinels with `NaN`, narrow the input types, or hoist the conditional out of the loop) before shipping.

These principles apply pervasively across the phases that follow. The phase descriptions list *what* to port; this subsection is *how* to port it.

15.2 Phase 1 — Foundations (~1 hour)

1. `Project.toml` + `Manifest.toml` with the dependency graph from §13 (including `CUDA.jl` and `KernelAbstractions.jl`).

2. `src/Types.jl` — the parametric types that encode the dimensional distinctions Python erases. Concretely:

   - `State{6} = SVector{6, Float64}` (from `StaticArrays.jl`) — stack-allocated, fixed-size, zero-overhead.
   - `DynParams = ComponentVector{Float64}` (from `ComponentArrays.jl`) — dot-access syntax (`theta.k_FB`) on a flat `AbstractVector` that `AdvancedHMC.jl` and `Enzyme.jl` accept directly.
   - `ParticleCloud = StructArray{Particle}` (from `StructArrays.jl`) — struct-of-arrays layout for coalesced GPU access.
   - `GPUFilterState{T}` and `CPUParameterCloud{T}` — the hybrid-execution type boundary from §14. The former wraps `CuArray` fields (state particles, weights); the latter is plain `Matrix{T}` / `Vector{T}`. Multiple dispatch chooses the right kernel based on which type is passed; the agent does not write two versions of any function.
   - Marker types for dispatch: `struct GaussianBridge end`, `struct SchrodingerFollmerBridge end`, `struct SoftSurrogate end`, `struct HardIndicator end`.

   All variable-length array fields use the abstract `AbstractArray{T,N}` where they could be either CPU (`Array`) or GPU (`CuArray`); fields that the hybrid contract pins to a specific device use the concrete type so the dispatcher can route correctly. This composite type-system is the LEAN4-charter-spirit applied to the framework.

3. `src/Config.jl` — translation of `core/config.py` `SMCConfig` as a `@kwdef` struct, plus a `backend::Symbol = :cpu` field (`:cpu` or `:cuda`) that selects the array constructor.

4. `src/Transforms.jl` — wraps `Bijectors.jl` for the unconstrained-space mapping.

15.3 Phase 2 — Filtering (~2 hours, GPU-resident)

Phase 2 is the inner SMC^2 loop and per the hybrid strategy of §14 it runs *on the GPU*: state particles are `CuArray`-resident and the kernels dispatch to CUDA via `StochasticDiffEq.jl` or `KernelAbstractions.@kernel`.

5. `src/Filtering/Kernels.jl` — Liu–West shrinkage moment match, Gaussian smoothing kernels, ESS computation. CUDA- dispatched via `AbstractArray{T,N}`-generic code.

6. `src/Filtering/Bootstrap.jl` — the inner bootstrap filter, producing $\widehat{L}_N(\theta_{\text{dyn}})$. The output is a single scalar (or a small vector if computing a gradient) that crosses back to the host CPU via PCIe; particle states stay on the GPU.

7. `src/Filtering/OT.jl` — wraps `OptimalTransport.jl` for the OT rescue; implements the sigmoid blend with the systematic-resample output.

8. Tests for Phase 2: differential against Python on a battery of random SDE+observation pairs. Tolerance 10^{-6} on per-step weights, 10^{-4} on integrated likelihood. **Tests run twice: once with `CPU()` arrays, once with `CUDA.CuArray`; both must pass within the same tolerance.**

15.4 Phase 3 — Outer SMC^2 (~1.5 hours, CPU-resident)

Phase 3 is the outer SMC^2 loop and per §14 it runs *on the CPU*: parameter cloud, log-likelihood vector, mass matrix, and HMC trajectories are all `Array`-resident. The likelihood callable provided to `AdvancedHMC.jl` performs a host-to-device transfer of the 37-D parameter vector and a device-to-host transfer of the scalar log-likelihood per gradient evaluation; both are microsecond-scale across PCIe Gen 5.

9. `src/SMC2/Tempering.jl` — adaptive $\delta\lambda$ via bisection on CPU. Direct port of `_solve_delta_for_ess`; the JAX `lax.while_loop` of the Python becomes a plain Julia `while` loop per §15.1.
10. `src/SMC2/HMC.jl` — wraps `AdvancedHMC.jl` for the rejuvenation moves on the CPU's branch-predictor-rich cores; re-estimates the diagonal mass matrix per tempering level. The likelihood closure dispatches into Phase 2's GPU filter for evaluation.
11. `src/SMC2/TemperedSMC.jl` — the outer loop, calling Phase 2 for likelihood, Phase 3.1–3.2 for tempering and HMC.
12. `src/SMC2/Bridge.jl` — Gaussian + BW-geodesic warm-starts, dispatched on marker types per §15.1.
13. Tests for Phase 3: differential against Python on the same benchmark `tests/test_obs_consistency_v5.py` already uses; both backends.

15.5 Phase 4 — Control (~1 hour)

14. `src/Control/Spec.jl` — `ControlSpec` struct (cost callable, decoder, horizon, initial state).
15. `src/Control/Calibration.jl` — $\beta_{\max}$ auto-calibration.
16. `src/Control/RBFSchedule.jl` — RBF schedule decoder.
17. `src/Control/TemperedSMC.jl` — re-uses Phase 3.3 with the cost-as-likelihood substitution.
18. Tests for Phase 4: differential against Python on the chance-constrained controller benchmark; both backends.

15.6 Phase 5 — Plant + Simulator (~0.5 hour)

19. `src/Simulator/SDEModel.jl` — wraps `StochasticDiffEq.jl`.
20. `src/Simulator/Observations.jl` — per-channel observation sampling.

15.7 Phase 6 — End-to-end + GPU benchmark (~1.5 hours)

21. Full-pipeline test: run a complete inference + control loop on the FSA-v5 model (consuming the LEAN4 reference from Part I), compare end-to-end against the Python pipeline on the same inputs. Run on **both backends** (`backend = :cpu` and `backend = :cuda`); confirm numerical agreement to within the same tolerances as Phase 2.
22. `JET.jl` static analysis pass; fix any flagged type uncertainties.
23. Benchmark per-window cost on CPU and GPU against the JAX-native Python baseline of ~ 1 second/window. Record the numbers in the package README; this is the artefact that justifies the port to anyone reading it later.

16 Scope and timing (framework side)

This is translation between two scientific computing languages. The user has explicitly rejected inflated estimates. With the existing Julia scaffolding from §13 doing the heavy lifting, the port is:

Phase	Estimate
Phase 1 — Foundations	1 hour
Phase 2 — Filtering (CPU + GPU)	2 hours
Phase 3 — Outer SMC^2 (CPU + GPU)	1.5 hours
Phase 4 — Control (CPU + GPU)	1 hour
Phase 5 — Plant + Simulator (CPU + GPU)	0.5 hour
Phase 6 — End-to-end + GPU benchmark	1.5 hours
Total	$\sim$ 7.5 **hours of focused work**

The CPU/GPU dual-target adds essentially nothing to the per-phase budgets above because Julia's generic dispatch on `AbstractArray{T,N}` means the same code runs on both `Array` (CPU) and `CuArray` (GPU); the library packages (`AdvancedHMC.jl`, `StochasticDiffEq.jl`, `OptimalTransport.jl`, `Distributions.jl`, `Bijectors.jl`) already have GPU support upstream. The marginal GPU cost is in (a) running the existing test suite a second time on `CuArray`, and (b) the Phase 6 benchmark — both already accounted for in the table.

For a developer competent in both languages, this is a single focused day's work. Most of it is wiring library calls together and running differential tests; the algorithmic content is not re-derived but transcribed. The resulting Julia is markedly shorter ($\sim$ 1,200 LOC vs $\sim$ 5,600 LOC) because the boilerplate that Python+JAX requires (closures, `Partial` wrappers, `lax.while_loop` scaffolding, manual `jit` placement) collapses in Julia.

The 7.5-hour estimate assumes an idiomatic, performance- optimised Julia port, not a line-by-line Python re-skin. A wooden translation that ports each `lax.scan` to a `Tullio` reduction and keeps Python-shaped control flow would technically compile in less time, but it would not realise any of the §12 benefits and would fail the §15.1 discipline. The estimate above is for code written the Julia way: plain `for` loops, multiple dispatch on marker types, `AbstractArray{T,N}` signatures, `StaticArrays.SVector` for fixed-size objects, `StructArrays` where the natural memory layout is struct-of-arrays, `Threads.@threads` or `Polyester.@batch` on outer particle loops, and `JET.jl`-clean type stability throughout. The performance-discipline patterns of §15.1 shape *how* the budgeted hours are spent; they do not add hours, because the right Julia idioms are typically shorter and faster than their Python+JAX equivalents (the $\sim$ 5,600 $\rightarrow\sim$ 1,200 LOC reduction is exactly that shrinkage). If the agent finds itself porting an algorithm faster than the table predicts, the result should still be checked against §15.1 — speed of writing is not the objective; an idiomatic, type-safe, GPU-portable, zero-allocation-inner-loop Julia codebase is.

The cost/value frame: a single 7.5-hour port amortises across *every future model the framework runs* (the JAX-native vs BlackJAX backend split disappears; the per-window recompile landmine disappears; the NUTS-inside-vmap cliff disappears; the agent pattern-match risk on framework internals disappears). The current trajectory — paying repeatedly for agent rounds of pattern-match-then-debug-Python+JAX-internals — is what this charter exists to stop.

17 What is and isn't covered (framework side)

In scope. A Julia package `SMC2FC.jl` providing filtering (bootstrap PF + OT rescue), outer SMC^2 (tempered + HMC + adaptive mass matrix + bridges), control-as-inference, an SDE simulator, and the unconstrained-space transforms. **Both CPU and GPU (CUDA) backends are in scope from day one**, and the production configuration is the **hybrid pipeline**

of §14: GPU for the inner particle filter, CPU for the outer SMC^2 rejuvenation. CPU-only and GPU-only configurations are also tested as diagnostic baselines. Differential tests against the Python implementation on the existing benchmarks for all configurations.

Out of scope (Python stays in service). The Python `smc2fc/` package continues to exist during and after the port. The Julia port does *not* replace it; it sits alongside as a type-checked alternative implementation. Existing user code that imports the Python package is unaffected.

Out of scope for the default port (settled in framework doc). The hard-vs-soft chance-constrained finding (use soft only) and the SF/BW-vs-Gaussian-bridge equivalence finding carry over from Python; the Julia port replicates the soft-HMC + Gaussian-bridge defaults.

Re-opened by the hybrid architecture (future work). The NUTS-as-HMC-replacement finding from framework-doc §6.4 is a *Python+JAX architectural* result: NUTS-inside-vmap-inside- `lax.while_loop` suffers warp divergence and dynamic-tree- expansion costs that combine for a 60–85× slowdown. The Julia hybrid pipeline of §14 runs NUTS on the CPU where its branchy control flow is well-matched to the hardware, which means *the empirical reason the framework rejected NUTS does not transfer to Julia.* The default Julia port still uses HMC for parity with the Python implementation, but a NUTS-on- hybrid benchmark is a candidate for the Phase 6 follow-on once the HMC port is green.

18 Audit checklist (framework-side agent)

A future agent runs this at session start, before touching any Julia file:

- ☐ Is there a Python reference for the function I am about to port?
- ☐ Does the LEAN4-first charter's process (Part I) apply (i.e. am I porting model code that should consume a LEAN4 spec)? If yes, does the spec exist?
- ☐ Have I declared parametric types for every input and output, capturing the dimensional information Python erases?
- ☐ Have I left array fields as `AbstractArray{T,N}` so the same code accepts `Array` (CPU) and `CuArray` (GPU)?
- ☐ Did I port the *algorithm*, not the *code*? Specifically: no `jax.lax.scan` / `lax.fori_loop` / `lax.while_loop` analogues survived the translation — they became plain Julia `for` loops. (Per §15.1.)
- ☐ Did I express type-dispatched behaviours via multiple dispatch (one method per marker struct) rather than as flag-driven conditionals carried over from Python? (Per §15.1.)
- ☐ Did I use `StaticArrays.SVector` for fixed-size small objects (`State{6}`, parameter blocks, RBF coefficient vectors) rather than `Vector`? (Heap-allocated `Vector`s in the inner loop regress versus JAX.)
- ☐ Did I use `StructArrays` or `ComponentArrays` where the natural memory layout is Struct-of-Arrays (particle clouds, parameter blocks)?
- ☐ Are inner loops zero-allocation? Verify with `BenchmarkTools.@btime`; an inner SDE step, Liu–West shrinkage step, or HMC leapfrog step should report `0 allocations` per iteration.

- ☐ Are outer particle loops parallelised on the CPU backend (`Threads.@threads` or `Polyester.@batch`)? On the GPU backend, replaced by a `KernelAbstractions` kernel?
- ☐ Does the function I just ported respect the hybrid contract from §14? Specifically: does inner-loop SDE/PF work stay on the GPU (`CuArray` fields), does outer-loop HMC/tempering work stay on the CPU (`Array` fields), and is the only host–device transfer the scalar log-likelihood (or the small parameter vector going the other way)?
- ☐ If I wrote a likelihood callable that `AdvancedHMC.jl` consumes, does it explicitly perform the host-to-device transfer on the parameter vector and the device-to-host transfer on the scalar log-likelihood, with nothing else crossing the bus per evaluation?
- ☐ Does `julia -project=. -e 'using Pkg; Pkg.test()'` pass on **both** the CPU and the CUDA test paths?
- ☐ Does `julia -project=. -e 'using JET; report_package(SMC2FC)'` report no type uncertainties *or* type instabilities? (Type instability is the #1 Julia performance killer; flag any `Union{...}` return in a hot inner loop.)
- ☐ Does the differential test against the Python implementation pass within the tolerance threshold for the function I just ported, on both backends?
- ☐ Have I updated the §15 port-mapping table to mark this file as ported?

If any box is unchecked, the agent does not proceed.

19 Closing of Part II

The point of Part II is accountability on the framework side. Python+JAX is a perfect storm for the kind of mathematical work `smc2fc` does: untyped arrays, invisible JIT contracts, closures over data, nesting cliffs. LLM agents pattern-matching through that surface burn the user's time and tokens hunting the consequences. Julia narrows the surface by structural means — a single command (`julia -project=. -e 'using JET; report_package(SMC2FC)'`) catches the kind of error that takes a human reviewer hours to find in the JAX/Python stack.

Combined with Part I, this charter establishes a single typed pipeline:

> **LaTeX equations** → **LEAN4 reference** (model side, Part I) → **Julia framework** (`smc2fc` side, Part II)
>
> *with Python remaining in service as the differentially-tested fast path on both sides.*

LEAN4 is never wrong about types. Julia narrows the dispatch surface to the point that pattern-matching does not silently produce wrong results. Python is fast and well-tested but pattern-match-friendly to a fault; it stays in service *as the fast path that gets checked against the typed reference*, not as the source of truth.

The total commitment in human hours — ~ 8 for Part I, ~ 7.5 for Part II, single-day-each scale — is the price of replacing repeated multi-day rounds of agent pattern-match-then- debug with a one-shot translation that holds for every future model and every future agent.